GNSS Sentinel

GNSS interference analysis
and receiver-telemetry classification

Signal processing / Machine learning / Evidence review

Contents

Select an entry to open the corresponding section.

Abstract 3
Problem context 4
Objectives and scope 5
Threat model and trust boundaries 6
Requirements and acceptance criteria 7
System architecture 8
Components and dependencies 9
Repository structure 10
End-to-end data lifecycle 11
Persistence and data model 12
Receiver telemetry schema 13
Validation and dataset provenance 14
Partitions and leakage control 15
Complex I/Q recording contract 16
Synthetic signal fixtures 17
Signal feature engineering 18
Spectrum and time-frequency analysis 19
Candidate classifiers 20
Preprocessing and class balance 21
Training and artifact lifecycle 22
Evaluation metrics 23
Saved synthetic experiment results 24
Confusion matrices and error analysis 25
Optional image classification 26
Inference and uncertainty handling 27
Normal-reference anomaly detection 28
Population shift monitoring 29
Incident evidence and review 30
HTTP API contracts 31
Training jobs and resource control 32
Dashboard workflow map 33
Security and failure handling 34
Installation and deployment 35
Verification strategy and evidence 36
Reproduction and operating runbook 37
Limitations and validity assessment 38
Technical development roadmap 39
References and implementation sources 40

Abstract

GNSS Sentinel is a modular platform for offline analysis of GNSS interference and receiver telemetry. It combines signal processing, supervised classification, optional image classification, distribution monitoring and incident review within a local application. Its central engineering objective is to make experiments traceable from input observations to stored model artifacts and reviewable predictions.

System contribution

The implementation extends a notebook-based starting point into a Python package with a shared command-line and HTTP interface. It adds explicit schemas, dataset fingerprints, reproducible partitions, training-only preprocessing, validation-based model selection and persistent experiment records. A browser dashboard exposes the principal workflows without requiring a notebook session.

Methods

Tabular experiments compare Random Forest, Extra Trees and logistic regression on common partitions. Complex I/Q recordings are summarized using twelve amplitude, phase and spectral features. The optional image workflow trains ResNet18 against explicit train, validation and test directories. Uncertainty, normal-reference anomalies and population shift provide additional evidence alongside a predicted class.

Evaluation

Saved synthetic experiments achieved test macro F1 values of 0.9798 for receiver telemetry and 0.9917 for signal features. These values describe generated fixtures, with 200 and 120 held-out observations respectively. The development verification record reports 30 passing tests, including a one-epoch image-training smoke test.

Interpretation

The platform demonstrates reproducible software workflows and structured analysis. It has not established detection performance on public recorded datasets or physical receiver captures. Field evaluation, calibrated alarm thresholds and capacity measurements remain separate technical requirements.

Keywords: GNSS, interference analysis, receiver telemetry, machine learning, signal processing, reproducibility, distribution shift.

Problem context

A useful GNSS analysis system must connect raw observations, experimental assumptions and decisions. A class label alone does not show whether the input was valid, whether the model encountered comparable data during training, or whether evaluation used independent captures. GNSS Sentinel treats these questions as part of the analysis workflow.

Availability and integrity

The project distinguishes interference classification from receiver-telemetry classification. The signal path examines patterns in complex samples; the telemetry path examines numerical receiver observations. They provide different evidence and use separate models. Their outputs are not fused into a measured positioning-integrity guarantee.

The reproducibility problem

Exploratory notebooks can distribute preprocessing, feature selection and evaluation across cells. Re-execution order, hidden state and copied datasets make comparisons difficult. The packaged implementation uses explicit inputs, fixed feature lists and saved experiment artifacts to make important decisions inspectable.

The operational interpretation problem

An unusual feature vector can arise from interference, receiver configuration, acquisition conditions or an unsupported input format. The interface therefore retains the top predicted class and additional review indicators. Incident status represents an analyst workflow rather than proof that an attack occurred.

Working terminology

A recording is a sequence of complex samples. A feature row summarizes a recording or receiver observation. A group denotes the independent acquisition unit supplied by the dataset author. A model is a fitted pipeline plus its metadata. An incident is a persistent record of an observation selected for review.

> The platform analyzes recorded observations. It does not generate radio transmissions, control a receiver, or estimate jammer location.

Objectives and scope

The project prioritizes a complete, inspectable analysis cycle: import, validate, partition, train, evaluate, infer and review. Scope is defined by executable modules and recorded verification rather than by the number of dashboard screens.

Research objective

Provide a repeatable baseline for comparing classifiers on explicitly identified data. Preserve a held-out partition, record the chosen candidate and expose class-level errors. Where capture identifiers exist, keep groups separate so that related observations do not cross evaluation boundaries.

Engineering objective

Centralize contracts and processing in reusable modules. Both the command-line interface and web application call the same training and data services. Keep durable metadata separate from large CSV and model files, while linking them with identifiers and checksums.

Analysis objective

Allow inspection of waveforms, spectrum summaries and spectrograms alongside numerical features. Retain uncertainty and anomaly indicators so an analyst can inspect observations that do not resemble the training reference.

Implemented scope	Boundary
Tabular classification	Two task schemas; local fitted pipelines.
Complex-signal analysis	Bounded I/Q CSV recordings, explicitly supplied sample rate.
Image workflow	Optional CLI training and checkpoint inference.
Monitoring and review	Batch drift diagnostics and incident status updates.
Deployment	One application process with local persistent storage.

> Distributed processing, continuous SDR capture, multi-tenant identity and field-certified detection are future work, not current capabilities.

Threat model and trust boundaries

The application accepts data that may be malformed and loads model artifacts from a trusted local registry. These are different trust boundaries. A numerical validation rule can reject invalid input, while a stored checksum can detect an unexpected artifact change; neither substitutes for control of the host operating system.

Assets

The principal assets are dataset contents, evaluation partitions, fitted model artifacts, experiment reports, API credentials and incident evidence. Integrity of the feature contract is particularly important: a column with the correct name but different units can produce a plausible yet misleading prediction.

Input boundary

CSV and JSON inputs are checked for required fields, finite numeric values and bounded sizes. Browser write requests with a foreign Origin are rejected. API-key checks are available for API routes, and the CLI requires a key when binding beyond loopback.

Artifact boundary

Dataset and model checksums are verified before sensitive processing. Artifact paths are constrained to the model storage area. Model files are generated locally; arbitrary serialized model uploads are not part of the API. A malicious actor able to alter both the registry and artifact can defeat the checksum assumption.

Residual risks

The implementation has no per-user authorization, cryptographic artifact signatures or tamper-evident incident ledger. The metadata database is an operational record, not an independent chain of custody. Host access, storage permissions and transport security remain deployment responsibilities.

Boundary	Accepted evidence	Important limit
CSV to dataset	Schema and fingerprint	Semantics still require review.
Registry to model	Trusted path and checksum	Registry itself must be trusted.
Prediction to incident	Stored model-linked evidence	Not an independently verified attack.

Requirements and acceptance criteria

Requirements are stated as observable behavior. A passing implementation test establishes that a contract is enforced for the exercised cases; it does not establish detection accuracy or production capacity.

Functional requirements

The system must register supported tabular data, compute features from bounded complex recordings, compare candidate models and retain test results. It must expose model inference, drift diagnostics and a reviewable incident list. Image classification must remain optional so the core application can run without the vision dependencies.

Quality requirements

Processing should reject invalid observations early, reuse the fitted preprocessing pipeline at inference and retain sufficient metadata to inspect a run. The application should bound training concurrency and request size. Errors should reach the caller without silently switching to a weaker evaluation protocol.

Requirement	Acceptance evidence
Schema integrity	Missing, duplicate and invalid training inputs are rejected.
Partition integrity	Group overlap is absent when group splitting is selected.
Preprocessing isolation	Scaler statistics originate only from training rows.
Artifact integrity	Changed dataset or model content causes a failure.
Reproducible inference	A reloaded pipeline predicts using its saved feature order.
Bounded orchestration	A full training queue rejects additional work.
Review lifecycle	Incidents can be listed, acknowledged and resolved.
Optional vision path	Checkpoint training, reload and prediction smoke test.

> No latency, throughput, availability or field-accuracy service-level target has yet been measured.

System architecture

GNSS Sentinel uses a single-process service architecture. The dashboard and command line are entry points into shared domain modules. Training runs in a bounded background executor; metadata lives in SQLite and larger artifacts remain in the configured runtime directory.

Shared service layer

The dataset service validates and registers observations. Signal utilities convert complex samples into features and visual summaries. Training services construct partitions, fit candidate pipelines and persist the selected model. Inference and monitoring use references saved from the training partition.

Persistence separation

SQLite stores searchable records and identifiers. CSV files and fitted pipelines are stored outside the database, with fingerprints recorded in metadata. This keeps record operations compact while allowing reports and data to be inspected independently.

Architectural consequence

A background thread avoids blocking the API handler while fitting a model, but it does not create a distributed scheduler. The queue, registry and filesystem assume one local application instance. A multi-node design would require coordinated job ownership and shared storage semantics.

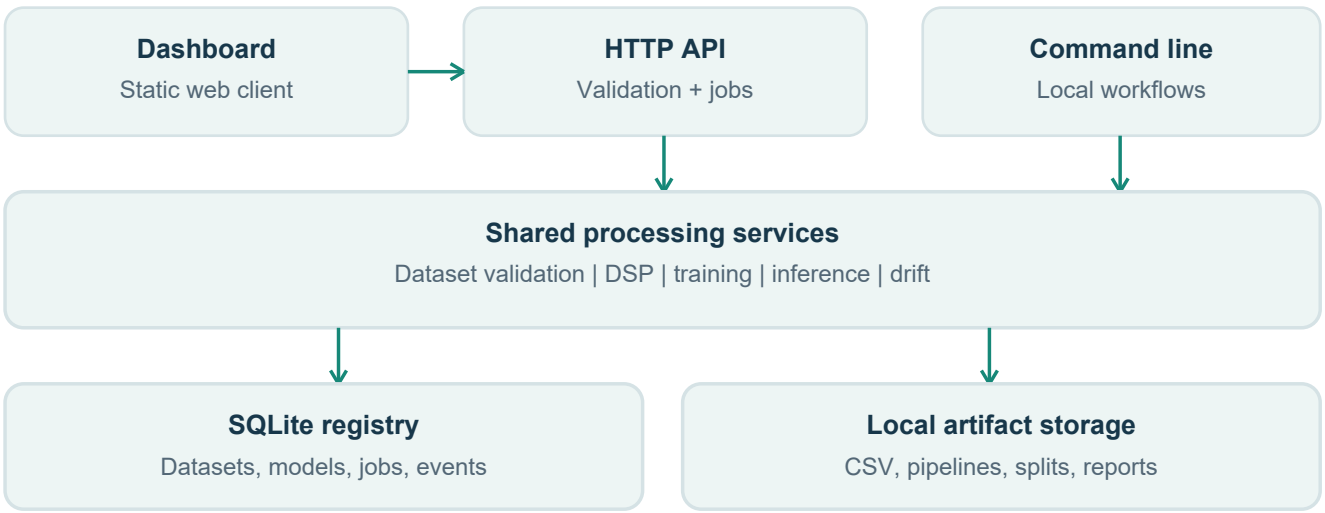


Figure 1. Implemented component relationships. The image workflow is a separate optional CLI path.

Components and dependencies

The Python package groups responsibilities by data lifecycle. HTTP handlers compose services rather than implementing independent versions of preprocessing or model training. The frontend uses static assets and does not require a JavaScript build pipeline.

Core processing

NumPy and SciPy provide complex-array and spectral operations. pandas handles CSV tables and feature selection. scikit-learn supplies preprocessing, classifiers, anomaly detection and evaluation utilities; joblib persists fitted tabular pipelines.

Application layer

FastAPI validates typed request models and exposes the service. Uvicorn runs the application. SQLite provides local record persistence. Configuration defines storage locations, upload limits and optional API authentication.

Optional dependencies

PyTorch and Torchvision are required for the image classifier. Keeping this path optional reduces the core installation footprint. Development dependencies provide the automated test client and test runner. Excel support is an optional import-related dependency, rather than a separate analysis engine.

Component	Primary implementation
Dataset contracts and fixtures	datasets.py
Complex-signal processing	signals.py
Tabular training and inference	training.py
Population shift	monitoring.py
Image training and prediction	vision.py
HTTP and orchestration	api.py
Persistence and configuration	store.py, config.py
Command-line entry point	cli.py

> Dependency ranges are specified in pyproject.toml. They are not an exact lockfile; reproducing an experiment also requires its recorded environment versions.

Repository structure

The repository retains the original research notebooks while placing reusable implementation code under src/gnss_sentinel. This separation makes the runnable application identifiable and preserves the provenance of the starting material.

Application package

The src layout exposes an installable package and CLI. The web directory supplies the HTML, CSS and JavaScript dashboard. The source catalog documents external datasets and integration notes without bundling those datasets into the repository.

Supporting material

The scripts directory contains launch, feature-extraction and telemetry-replay utilities. Tests exercise processing contracts and service behavior. Documentation covers architecture, verification and operation; the report summarizes those elements as a connected system.

Runtime artifacts

The runtime directory is excluded from version control. A deployment creates its own SQLite database, registered CSV files and model artifacts. A source checkout therefore does not imply that trained models or externally acquired recordings are present.

```
src/gnss_sentinel/
  api.py          cli.py          config.py
  datasets.py     signals.py      training.py
  monitoring.py   store.py        vision.py
  web/            catalog.json
scripts/          tests/          docs/
docs/reports/GNSS-Sentinel-Project-Report.pdf
pyproject.toml    Dockerfile      compose.yaml
runtime/        # generated local data; excluded from Git
```

> Read the original notebook directories as research material. Use the packaged CLI and documented scripts for the reproducible platform workflow.

End-to-end data lifecycle

A run begins with a source declaration and a feature contract. Registration creates a canonical dataset representation and stores its fingerprint. Training references that immutable content, while inference references a specific saved model.

1. Acquire and interpret

Identify the source, license, label semantics and acquisition unit before import. For telemetry, verify the required feature names and units. For I/Q data, provide the sample rate and convert the recording explicitly into I and Q columns. Assign groups from capture metadata rather than from a desired train/test balance.

2. Register and evaluate

Validation rejects unsupported values and ambiguous training rows. The registered CSV receives a SHA-256 fingerprint. Training checks that fingerprint, constructs partitions, fits preprocessing and candidate classifiers, then records the selected model and held-out metrics.

3. Analyze new observations

Prediction validates the model-specific feature schema and uses the saved preprocessing pipeline. The response includes a predicted class, confidence-like score and review indicators. Drift analysis compares a batch with training-only feature histograms.

4. Review and preserve

When enabled, selected predictions create incidents containing model-linked evidence. Analysts can update their review status and export records. A reproducible investigation retains the input source, registered dataset, model report and relevant incident identifiers together.

Stage	Durable output
Registration	Dataset record, canonical CSV and content hash.
Training	Split indices, pipeline, validation scores and test report.
Inference	Response evidence; optional incident record.
Review	Incident status and exported evidence.

> A dataset identifier points to registered content; a model identifier points to a fitted experiment. They should not be treated as interchangeable version labels.

Persistence and data model

The registry uses a compact SQLite records table. Each record has a kind, identifier, creation value and JSON body. This supports the current small set of entity types without requiring a separate relational table for every report structure.

Record categories

Datasets describe registered observations and provenance. Models reference fitted artifacts and evaluation results. Jobs track asynchronous training state. Events store incident evidence and review status. An index over kind and creation order supports recent-record queries.

Transaction behavior

Connections are opened for operations and closed afterwards. Write-ahead logging supports the local access pattern; updates use a write transaction. The store is an application registry rather than an event-sourcing system, so an updated record does not automatically preserve every previous version.

File coordination

Model folders contain pipeline.joblib, report.json and splits.json. Registration links metadata only after artifacts have been written. Dataset contents use a hash-based CSV name. Database and filesystem backups must therefore be treated as one coordinated snapshot.

Recovery implications

An incomplete training process can leave a failed job or unused artifact directory. Startup marks formerly queued or running jobs as failed. It does not resume partially fitted estimators. Operators should investigate failures and explicitly submit a new run.

Entity	Key information
Dataset	Task, features, classes, rows, source, fingerprint.
Model	Algorithm, seed, metrics, artifact references, checksum.
Job	Dataset request, state, result or failure information.
Event	Sensor, source, model, prediction, severity, status.

> Back up both the registry and artifact directories. Copying only the database cannot restore fitted pipelines or registered observations.

Receiver telemetry schema

The spoofing task expects thirteen numeric features in a fixed contract. Column names alone do not establish compatible units or receiver semantics. Dataset documentation must resolve those details before a model trained on one source is applied to another.

Feature order

The required sequence is PRN, DO, PD, RX, TOW, CP, EC, LC, PC, PIP, PQP, TCD and CN0. The training artifact preserves the feature list so inference can select the same columns. The application does not infer alternate names or automatically reconcile receiver-specific definitions.

Labels and groups

Training accepts label, or the legacy Output column as a fallback. Labels are converted to trimmed strings. An optional group column represents sessions or acquisition units. The synthetic fixture uses authentic, simplistic, intermediate and sophisticated; imported sources can have other class names within the validation limits.

Semantic review

Before import, confirm what each label means, whether labels describe an entire session or an individual row, and whether measurements are synchronized. Features such as identifiers or observation time can correlate with a capture session; good row-level accuracy may therefore reflect a recording shortcut rather than a useful interference pattern.

Field category	Contract
Required observations	13 named numeric columns; finite and within accepted range.
Training target	label or Output; 2-20 distinct classes.
Class support	At least 10 rows per class before partitioning.
Acquisition unit	Optional nonempty group values.
Prediction input	Required numeric features; no training label required.

> Normal-reference modeling recognizes authentic, clean, 0 and 0.0. Other encodings require deliberate label mapping before relying on anomaly behavior.

Validation and dataset provenance

Validation is a gate before registration, not a repair routine that silently changes the meaning of a dataset. The system rejects incompatible content so the caller can correct the source or document a transformation explicitly.

Numerical and structural checks

Required features must exist and be numeric. Values must be finite and have absolute magnitude no greater than 10^15. Empty inputs, empty labels and labels longer than 80 characters are rejected. Training requires enough observations per class to support the evaluation workflow.

Duplicate protection

Exact duplicate feature rows are rejected during training registration, including rows with different labels or groups. This catches a simple leakage route and contradictory annotations. It does not detect near duplicates, overlapping signal windows or semantically related samples; those need capture-aware grouping.

Provenance capture

The canonical CSV is fingerprinted with SHA-256. Metadata records source description, task, feature names, class counts and group information. A matching checksum shows that the registered bytes have not changed relative to the trusted record; it does not independently verify the stated acquisition source.

Import limits

Browser CSV imports are bounded to 20 MiB and 150,000 rows. Command-line imports avoid that HTTP limit but still load data into memory. Scaling file size through the CLI is therefore not equivalent to streaming ingestion.

Failure	Expected response
Missing or nonnumeric feature	Reject and identify the incompatible input.
Nonfinite or extreme value	Reject before fitting or prediction.
Duplicate training features	Reject instead of silently deduplicating.
Changed registered CSV	Fail checksum verification before training.

> Keep the original source separately from the canonical registered CSV and document any unit conversion, label mapping or filtering.

Partitions and leakage control

Evaluation separates training, validation and test observations before fitting transformations. The target allocation is approximately 70/15/15, but group splitting allocates acquisition units rather than individual rows. Exact row counts consequently depend on group size.

Grouped path

When group identifiers are supplied, the workflow requires at least six distinct groups. A seeded GroupShuffleSplit assigns about 70% of groups to training and divides the remainder between validation and test. Every partition must contain every class. A failed grouped split raises an error instead of silently switching to a row split.

Ungrouped path

Without group metadata, the system uses stratified row splitting and records that capture independence is unverified. Stratification maintains class representation; it does not prevent observations from the same recording appearing in different partitions.

Preprocessing boundary

The scaler and each candidate classifier fit only on training rows. Validation selects the candidate, and the test set measures the selected pipeline. The winner is not refitted on combined training and validation data. This keeps the saved fitted object aligned with the reported experiment. [4]

What grouping cannot prove

Artificially assigned groups do not create acquisition independence. Adjacent windows from the same recording should share an appropriate parent identifier. For real data, receiver, capture and session metadata must be reviewed before interpreting generalization.

Saved fixture	Train	Validation	Test
Telemetry: 1,200 rows / 30 groups	840	160	200
Signal features: 600 rows / 10 groups	420	60	120

> Persisted split indices make the selected observations inspectable. A fixed seed improves repeatability but cannot repair an invalid grouping strategy.

Complex I/Q recording contract

A complex recording is represented as x[n] = I[n] + jQ[n]. The upload format uses explicit I and Q CSV columns and a separately supplied sample rate. The system does not guess binary layout, endianness, receiver tuning or missing acquisition metadata.

Accepted signal domain

Validation requires a one-dimensional complex array with 256 to 131,072 samples. Samples must be finite, nonzero as a recording and bounded in magnitude. Accepted sample rates range from 1 kHz to 100 MHz. The numerical bounds protect processing; they do not define the bandwidth of a particular GNSS receiver.

Two processing lengths

Feature extraction uses the entire accepted recording. Time-frequency visualization is limited to the first 16,384 samples, and the displayed waveform uses the first 256. A plot can therefore omit later behavior that still contributes to the model features. The distinction matters when investigating transient events.

Manifest extraction

The extraction script reads a manifest containing path, label, group and sample rate. It converts each listed I/Q CSV recording into one feature row. Label and group assignment remain the responsibility of the capture manifest, rather than being inferred from the signal.

Measurement interpretation

Input amplitude and spectral power are relative to the sample representation. No antenna, receiver gain or calibration chain is modeled. Values should not be described as calibrated dBm or converted to absolute radio-frequency power without independent calibration.

Input property	Required interpretation
I and Q	Paired numerical components in acquisition order.
Sample rate	Explicit sampling frequency in hertz.
Label	Known annotation for supervised training.
Group	Independent capture/session identifier.

> Preserve original capture metadata even if only I/Q columns are imported. Sampling and normalization choices affect the meaning of extracted features.

Synthetic signal fixtures

Synthetic fixtures provide repeatable examples for exercising feature extraction, plots and classification. They contain six pattern families: clean, narrowband, chirp, pulsed, amplitude-modulated and frequency-modulated. Their labels describe the generator, not a verified physical transmitter.

Construction

The default generator produces 4,096 complex samples at 2 MHz. It combines a normalized pattern with complex noise, using a seeded random generator. The default SNR argument is 10 dB; the classification fixture samples a broader range from -5 to 20 dB. Frequency and modulation parameters vary across generated examples.

Pattern interpretation

Narrowband examples concentrate energy near a tone. Chirps vary instantaneous frequency. Pulsed examples gate a tone in time, while AM and FM vary amplitude or phase behavior. The clean class is noise-only and does not contain a modeled authentic GNSS spreading waveform.

Fixture design limit

The signal fixture assigns synthetic group identifiers and produces 100 examples per class. This tests the mechanics of group separation but does not represent independent field capture sessions. A generated pulsed waveform is also not an exact implementation of a DME signal.

Class	Generator characteristic
clean	Complex noise baseline.
narrowband	Single complex tone with noise.
chirp	Time-varying tone frequency.
pulsed	Gated tone pattern.
am	Tone with sinusoidal amplitude variation.
fm	Tone with sinusoidal frequency modulation.

> High fixture accuracy can reflect deliberately distinct generator families. It cannot be transferred directly to a real receiver environment.

Signal feature engineering

The tabular jamming path reduces a complex recording to twelve scalar features. The spectral branch applies a Hann window, computes the centered FFT and normalizes squared magnitudes into bin proportions p[k]. The features combine energy concentration, envelope shape and phase variation.

Normalization conventions

Spectral entropy is divided by log(N), where N is the number of bins. Centroid, spread and occupied bandwidth use normalized frequency in cycles per sample. They are not expressed directly in hertz. Small numerical offsets prevent logarithms and divisions from becoming undefined.

Interpretation

Features summarize a window and can hide event order. Two signals with similar global spectral and envelope statistics can still differ in their time-frequency evolution. Spectrogram inspection complements this compact representation but is not automatically fused with the tabular classifier.

Feature	Definition or operational meaning
power_db	10 log10(mean |x|^2 + epsilon).
crest_factor	Maximum amplitude / root-mean-square amplitude.
spectral_entropy	-sum p log(p) / log(N).
spectral_flatness	Geometric mean spectral power / arithmetic mean.
peak_fraction	Largest normalized spectral bin.
occupied_bandwidth	Frequency at 99% cumulative power minus that at 1%.
envelope_cv	Standard deviation / mean of amplitude.
phase_step_std	Spread of angle(x[n+1] conjugate(x[n])).
duty_cycle	Fraction of amplitudes above half the maximum.
spectral_centroid	sum f[k] p[k].
spectral_spread	sqrt(sum (f[k] - centroid)^2 p[k]).
envelope_kurtosis	Fourth central moment / squared variance; not excess.

> The duty-cycle value is an amplitude-threshold proxy. It is not a validated pulse-duration estimator.

Spectrum and time-frequency analysis

Signal Lab derives a time-frequency representation from a centered short-time FFT. SciPy ShortTimeFFT supplies the windowed transformation; the implementation uses a 256-sample periodic Hann window and a 128-sample hop. Frequency and time axes are returned with the numerical display data. [5]

Resolution tradeoff

At the default 2 MHz sampling rate, a 256-sample FFT has a 7.8125 kHz bin spacing and a 128-sample hop advances by 0.064 ms. These are analysis-grid spacings, not guaranteed resolution of closely spaced physical sources. Window shape and signal duration influence what can be distinguished.

Displayed quantities

The heatmap uses 10 log10 of squared STFT magnitude with a small offset. The spectrum summary takes mean power across time before applying the logarithm. Averaging powers and averaging logarithmic values are different operations; the implemented order should be preserved when reproducing a plot.

Investigation procedure

Confirm sample rate first, then inspect whether energy persists, sweeps or appears intermittently. Compare the plotted excerpt with the full recording length. Use the numerical features to relate the visual pattern to the classifier input, while retaining the source recording for later review.

Limits

The frontend representation is a visualization, not a calibrated spectrum analyzer. The processing does not estimate absolute RF frequency because a tuning frequency is not part of the transform contract. Cropping the display to 16,384 samples bounds plotting work but may exclude later transients.

Parameter	Value or rule
Window / hop	256 / 128 samples.
FFT arrangement	Centered frequency axis.
Display duration limit	First 16,384 samples.
Axes	Frequency in kHz; time in ms.
Feature extraction length	Entire accepted recording, independently of plot crop.

Candidate classifiers

Each tabular experiment compares three classifier families on the same train, validation and test partitions. Fixed configurations establish an inspectable baseline. The current process is a candidate comparison, not an exhaustive hyperparameter search.

Random Forest

A forest of 140 trees uses balanced class weights and a minimum leaf size of two. Two worker threads are requested for estimator fitting. The method can model nonlinear relationships and feature interactions without a handcrafted polynomial expansion.

Extra Trees

The Extra Trees configuration also uses 140 trees, balanced class weights and a minimum leaf size of two. Its randomized splitting procedure provides a different ensemble baseline. The validation result determines whether it is retained for a given task.

Logistic regression

The linear baseline uses balanced class weights and a maximum of 1,500 optimization iterations. Standardization is particularly important because feature scales differ. Including this candidate helps reveal whether a complex model materially improves on a simpler decision surface.

Comparison discipline

All candidates receive the same observations and feature order. Selection uses validation macro F1 with a strict improvement test; an exact tie retains the earlier candidate. The test partition does not choose the model. No claim of optimal architecture follows from winning this limited comparison.

Setting	Tree candidates	Linear candidate
Class weights	Balanced	Balanced
Size / iterations	140 trees	1,500 max iterations
Minimum leaf	2	Not applicable
Random seed	Recorded	Recorded
Selection metric	Validation macro F1	Validation macro F1

> The saved experiment metadata is authoritative for a run. Configuration changes require a new experiment rather than rewriting an earlier result.

Preprocessing and class balance

Every candidate is wrapped in a fitted preprocessing pipeline. The StandardScaler learns location and scale from training rows and applies the same transformation to validation, test and later inference observations. The fitted transformer is persisted with the classifier.

Why a shared pipeline matters

A feature may have a large numeric range without carrying more useful information. Scaling makes the linear baseline less dependent on units and prevents inference code from rebuilding normalization independently. Tree models do not generally need this scaling, but the uniform wrapper keeps the processing contract explicit.

Class weighting

Balanced class weights adjust fitting to class frequencies in the training partition. They do not rebalance the held-out test set or change the reported support counts. A balanced synthetic fixture can still hide the challenges of a deployment where unusual observations are rare.

No silent imputation

Missing or invalid numeric inputs are rejected. The pipeline does not learn an imputer, remove outliers or synthesize minority examples. Adding any of these steps would require training-only fitting and its own evaluation because it changes the input distribution.

Reference statistics

The inference artifact also retains training medians and standard deviations for deviation checks. These values serve a separate diagnostic role from the scaler. A small floor protects division for nearly constant features. A large normalized deviation is evidence of input mismatch, not an explanation of an attack mechanism.

Operation	Data allowed to influence it
Scaler fitting	Training partition only.
Classifier fitting	Training partition only.
Candidate selection	Validation metrics.
Reported held-out score	Test partition after selection.
Drift reference and deviation statistics	Training observations.

> Reusing the saved pipeline prevents a common mismatch between training-time normalization and prediction-time input handling. [4]

Training and artifact lifecycle

The training function resolves a dataset record, checks the stored CSV fingerprint and validates its task schema. It creates partitions before fitting and saves sufficient information to inspect both the selected model and the comparison that produced it.

Candidate fitting

The scaler and classifier for each candidate fit on training observations. Validation predictions produce accuracy and macro F1. The highest validation macro F1 identifies the selected fitted pipeline; an exact tie keeps the first candidate that achieved the score.

Held-out evaluation

The selected pipeline predicts the test partition once within that run. Accuracy, balanced accuracy, macro F1, a confusion matrix and per-class precision, recall and F1 are retained. Repeated manual changes informed by test results would still contaminate the experiment, even though the code separates the arrays.

Artifact contents

pipeline.joblib contains the fitted pipeline and supporting references. splits.json records partition indices. report.json captures the experiment report. A model record links these outputs with feature names, seed, algorithm and version metadata. The artifact fingerprint is checked before later inference.

Failure behavior

A validation, split or fitting failure should be investigated at its originating stage. The workflow does not register a successful model before artifact writing completes. Failed asynchronous jobs retain their failure state rather than appearing as completed experiments.



Figure 2. Selection uses validation data; the final evaluation uses the held-out test partition. The selected pipeline is saved without a train-plus-validation refit.

Evaluation metrics

The report uses complementary classification metrics because aggregate accuracy can conceal minority-class failures. All stored test metrics on this page refer to the selected classifier before inference-time abstention and incident rules are applied.

Per-class precision and recall

For a class treated as positive, precision = TP / (TP + FP) and recall = TP / (TP + FN). Precision asks how often a prediction is correct; recall asks how much of the actual class was recovered. Their interpretation depends on the prevalence and labeling of the evaluated data.

F1 and macro averaging

F1 = 2PR / (P + R). Macro F1 is the unweighted mean of class F1 values, so each class contributes equally regardless of support. It is the validation selection criterion. It should be read alongside class supports and a confusion matrix rather than as a complete operational measure.

Accuracy and balanced accuracy

Accuracy is the number of correct predictions divided by all evaluated observations. Balanced accuracy averages class recalls. With equal class supports, these can coincide, as they do in the stored synthetic test partitions.

Operational evaluation still needed

A deployment study should separately measure false alarms, missed detections, abstention frequency and review workload on representative capture groups. Confidence calibration and interval estimates require a designed evaluation protocol. The current saved run reports point estimates, not uncertainty intervals or field operating guarantees.

Quantity	Question answered
Confusion matrix	Which true classes are confused with which outputs?
Per-class support	How many evaluated examples support each score?
Macro F1	How well are classes handled with equal importance?
Balanced accuracy	What is the average recall across classes?
Incident rate	Not established by the classifier metrics alone.

Saved synthetic experiment results

The figures below are read from the saved local model reports. Both experiments use seed 42 and record scikit-learn 1.9.1. No external dataset was used for these results. Candidate scores describe validation predictions; final scores describe the selected pipeline on its test partition.

Telemetry experiment

Extra Trees was selected. Its validation macro F1 tied logistic regression at approximately 0.9685, and the earlier candidate was retained. The test partition contains 200 observations with 50 examples per class. Four predictions are incorrect, all from the sophisticated fixture class.

Signal-feature experiment

Random Forest achieved validation macro F1 of 1.0000 and was selected. Its test partition contains 120 observations with 20 per class. One FM example was classified as AM. The group-based split produced a smaller validation partition than the nominal 15% row target.

Meaning of the scores

These runs establish that the full software path can fit, compare, persist and evaluate real estimators on generated data. They do not show robustness to receiver changes, unfamiliar interference, acquisition artifacts or class imbalance in operational recordings.

Measure	Telemetry	Signal features
Selected model	Extra Trees	Random Forest
Train / validation / test	840 / 160 / 200	420 / 60 / 120
Validation RF macro F1	0.9623	1.0000
Validation ET macro F1	0.9685	0.9833
Validation LR macro F1	0.9685	0.9663
Test accuracy	0.9800	0.9917
Test balanced accuracy	0.9800	0.9917
Test macro F1	0.9798	0.9917

> Recorded per-run durations are not used as capacity benchmarks. Hardware, workload and repeat measurements would be needed for that purpose.

Confusion matrices and error analysis

The matrices show counts, with true classes on rows and predicted classes on columns. They describe classifier outputs on synthetic held-out observations. They do not include the additional uncertain label, normal-reference anomaly rule or incident creation policy.

Telemetry errors

Of 50 sophisticated examples, 46 retain that label, three are assigned authentic and one simplistic. The sophisticated class therefore has recall 0.92. Perfect recall for the other three fixture classes does not establish that those classes are generally easy on recorded data.

Signal-feature error

The sole signal error is an FM-to-AM prediction. This identifies a useful fixture example for inspection of envelope and phase statistics. One example is insufficient to establish a systematic physical confusion mechanism.

Next diagnostic step

Preserve the relevant test indices and compare error rows against the training references. On real recordings, inspect errors by capture group, receiver and acquisition conditions. Repeated model changes should use validation data and a newly protected final evaluation design.

Telemetry

Rows: true class | Columns: predicted

	Auth.	Inter.	Simple	Soph.
Auth.	**50**	0	0	0
Inter.	0	**50**	0	0
Simple	0	0	**50**	0
Soph.	**3**	0	**1**	**46**

n = 200 held-out synthetic observations

Signal features

Rows: true class | Columns: predicted

	AM	Chirp	Clean	FM	NB	Pulse
AM	**20**	0	0	0	0	0
Chirp	0	**20**	0	0	0	0
Clean	0	0	**20**	0	0	0
FM	**1**	0	0	**19**	0	0
NB	0	0	0	0	**20**	0
Pulse	0	0	0	0	0	**20**

n = 120 held-out synthetic observations

Figure 3. Stored test confusion matrices; telemetry n = 200 and signal features n = 120. Counts are taken from the saved reports.

Optional image classification

The image workflow uses Torchvision ResNet18 with a replacement final classifier. It is invoked through the CLI and writes its own checkpoint and reports. It is not currently connected to the tabular model registry or dashboard training queue. [6]

Dataset contract

The root contains train, val and test directories using ImageFolder class subdirectories. Each partition must have the same class mapping and at least two classes. Content hashes reject identical images crossing partitions. Near duplicates and images derived from overlapping capture windows still require source-aware split design.

Preprocessing and optimization

Images are converted to RGB, resized to 128 by 128 pixels and normalized with ImageNet channel statistics. Training uses AdamW with learning rate 0.0002 and weight decay 0.0001, cross-entropy loss and gradient-norm clipping at one. Pretrained weights are optional and downloaded only when requested.

Selection and prediction

Validation macro F1 selects the best checkpoint. That checkpoint is reloaded before test evaluation. Prediction uses evaluation mode and disables gradient computation. The workflow records the image manifest and class mapping so the checkpoint can be interpreted with its original label order.

Verification boundary

The recorded smoke test used random images for one epoch. It exercised optimization, persistence, reload and inference rather than testing meaningful visual classification. No public image-dataset performance result is claimed.

```
gnss-sentinel train-images data/images --output runs/vision-01
gnss-sentinel predict-image runs/vision-01/best.pt sample.png
```

> The output directory must be new. An image label is only as reliable as its source annotation and partition provenance.

Inference and uncertainty handling

Prediction loads a locally registered model only after checking its artifact path and checksum. Input rows are validated against the saved feature contract, then passed through the fitted pipeline in the recorded feature order.

Top class and returned label

The classifier supplies per-class probabilities and a top class. The returned label becomes uncertain when the maximum probability is below 0.55 or any feature lies more than eight training standard deviations from its training median. The top class remains available for inspection.

Scores are not guarantees

The probabilities are not calibrated in the current workflow. A maximum value of 0.9 therefore should not be presented as a verified 90% chance of correctness. The uncertainty thresholds are heuristics requiring validation against representative independent captures.

Deviation evidence

The response includes the largest feature deviations from the training reference. These describe how the observation differs numerically. They are not a causal attribution method and should not be interpreted as proving which physical process produced the observation.

Bounded requests

The prediction API accepts one to 512 rows per call. Batching amortizes interface overhead, but the implementation does not provide a measured streaming deadline. Clients should retain their own observation identifiers and distinguish live acquisition from replayed data.

Condition	Returned interpretation
High score and within reference bounds	Selected class, with available supporting evidence.
Maximum score below 0.55	uncertain; retain top_class for inspection.
Any normalized deviation above 8	uncertain; report the outlying features.
Invalid schema or artifact mismatch	Reject; do not produce an ordinary prediction.

> The saved classifier confusion matrices precede these uncertainty rules. End-to-end abstention behavior needs a separate evaluation.

Normal-reference anomaly detection

A separate anomaly model can be fitted when the training partition contains at least twenty observations recognized as normal. It uses normal training rows rather than the whole labeled dataset, providing a reference distinct from the multiclass classifier.

Eligibility and fitting

Recognized normal labels are authentic, clean, 0 and 0.0. Eligible rows fit a scaler followed by an Isolation Forest with 100 estimators, contamination 0.05 and the experiment seed. Other label conventions must be deliberately mapped; the implementation does not infer their meaning.

Returned evidence

The Isolation Forest decision function provides an anomaly score. Negative values are flagged by the current rule. This can identify observations that differ from the learned normal reference even when the multiclass classifier favors a normal label.

Interpretation of contamination

The contamination parameter controls the fitted scoring threshold. It is not a measured real-world attack prevalence and should not be described as a guaranteed false-alarm rate. Threshold behavior changes with the representativeness of the normal training observations.

Evaluation needs

A useful study would isolate verified normal captures across receiver conditions, evaluate false alarms by group and measure sensitivity on held-out unusual observations. The current synthetic classifier report does not independently benchmark anomaly detection or quantify its contribution to incident volume.

Supervised classifier	Normal-reference model
Fits all labeled training classes.	Fits recognized normal training rows.
Returns class probabilities and top class.	Returns a decision-function score.
Uses validation to choose a candidate.	Uses a fixed Isolation Forest configuration.
Has saved synthetic class metrics.	Has no separate field-performance benchmark.

> A normal-reference anomaly is a reason to inspect an observation. It does not identify the cause or replace the supervised class interpretation.

Population shift monitoring

The drift monitor compares a new batch with feature distributions saved from the training partition. It calculates population stability index (PSI) separately for each feature and also reports mean displacement relative to the training spread.

Reference construction

Training quantiles define interior bin edges; duplicate edges are collapsed and the outer bounds extend to negative and positive infinity. The saved reference contains expected proportions. Current counts receive smoothing so empty bins do not create undefined logarithms.

PSI calculation

For bin proportions a[i] in the current batch and e[i] in the reference, PSI = sum((a[i] - e[i]) ln(a[i] / e[i])). Matching proportions produce zero. Larger values indicate stronger distribution disagreement under the chosen binning, but do not identify its cause.

Operational bands

The implementation uses 0.10 and 0.25 as heuristic boundaries between lower, moderate and higher shift. These thresholds have not been calibrated for GNSS receiver behavior. Batch size, correlated observations and feature dependence all affect interpretation.

Use in review

The API accepts 50 to 5,000 observations for a drift batch. A shift can reflect receiver configuration, environmental conditions, acquisition changes or a different population. Investigate provenance and feature semantics before retraining. Automatically replacing a model after a drift flag is not part of the current implementation.

Observed situation	Reasonable follow-up
One feature shifts strongly	Check units, parsing and receiver configuration.
Many features shift together	Check source population and acquisition conditions.
Small or highly correlated batch	Collect a more representative evaluation batch.
Persistent shift with labels available	Design an independent validation and retraining study.

> Drift is a batch distribution diagnostic. It is neither a per-row class label nor proof of adversarial activity.

Incident evidence and review

The incident workflow retains selected predictions for human inspection. With incident recording enabled, a non-normal returned label or a flagged anomaly can create an event linked to the model and input source.

Evidence fields

An event includes sensor identity, source information, model identifier, prediction evidence, severity and review status. The uncertain label receives review severity; other triggered cases use high severity. This prioritization is an application convention rather than a calibrated estimate of operational harm.

Status lifecycle

Events begin open. The API supports acknowledged and resolved statuses so an analyst can track review. A resolved status records workflow progress; it does not itself validate the predicted class or establish a physical explanation.

Listing and export

The events endpoint retrieves a bounded recent set. Status filtering occurs after that retrieval, so a filtered response should not be assumed to enumerate every matching historical event. CSV export escapes leading text characters that spreadsheet software could interpret as formulas.

Replay provenance

The telemetry replay script submits bounded batches and identifies observations as replayed input. Replayed records should remain distinguishable from newly acquired measurements. Synthetic simulations likewise need to retain their origin when interpreted in the incident view.

Review step	Evidence to inspect
Identify	Model, source, sensor and observation context.
Check validity	Feature semantics, uncertainty and anomaly indicators.
Compare	Training reference and related capture observations.
Record outcome	Status update and retained supporting material.

> The current store does not provide an immutable audit trail of every status transition. Evidence-grade history would need an explicit append-only design.

HTTP API contracts

FastAPI exposes typed request models and serves the dashboard from the same application. Interactive API documentation is available at /docs. API consumers should follow the generated schema for exact field names and response structures.

Resource conventions

Dataset identifiers select registered content; model identifiers select fitted artifacts. Public metadata omits internal filesystem paths. Input validation occurs before domain processing, and training submission returns a job record rather than waiting for fitting to finish.

Client behavior

Clients should treat a rejected input as a failed operation and preserve the validation response. A full training queue produces a rejection rather than accepting unlimited work. Prediction and monitoring calls have explicit batch limits and should be split accordingly.

Method and route	Purpose
GET /health	Service health response.
GET /api/overview	Counts and recent models, jobs and incidents.
POST /api/datasets/upload	Register a supported labeled CSV.
GET /api/datasets/{id}/sample	Preview registered observations.
POST /api/experiments	Enqueue a training experiment.
GET /api/models	List model metadata.
POST /api/predict	Predict a bounded feature-row batch.
POST /api/signals/upload	Analyze a bounded I/Q CSV recording.
POST /api/signals/simulate	Generate and inspect a synthetic pattern.
POST /api/drift	Compare a batch with a training reference.
PATCH /api/events/{id}	Update an incident review status.
GET /api/events/export	Export stored incident evidence as CSV.

> API-key protection applies to /api/ routes when configured. Health, static assets and API documentation are outside that key gate.

Training jobs and resource control

Training is scheduled within the application process. A single-worker executor performs one training job at a time, while a bounded semaphore limits accepted work to three jobs: one active and at most two waiting.

State transitions

A submission enters queued, then running when execution begins. Successful completion records the resulting model; an exception records failed. Job records allow clients to inspect progress without holding the original HTTP request open.

Backpressure

When the accepted-work capacity is full, the endpoint rejects additional submissions with HTTP 429. A client should wait for available capacity before retrying. The queue bound controls accepted training tasks; it does not guarantee a fixed CPU or memory limit for each estimator.

Restart and shutdown

Startup marks previously queued or running records failed because the in-memory execution state cannot be resumed. Shutdown waits for the executor. There is no durable task broker, worker lease, checkpointed tabular fitting or distributed cancellation protocol.

Scaling implication

Adding web workers would create multiple local executors and alter the concurrency assumptions. The supported CLI starts one worker. A larger deployment should move jobs into an external queue, define idempotent submission and result registration, and measure memory use under concurrent workloads.

State	Meaning	Next normal state
queued	Accepted; waiting for executor.	running
running	Training function is executing.	completed or failed
completed	Model result is available.	Terminal
failed	Run interrupted or rejected by processing.	Terminal; resubmit explicitly

> Bounded concurrency is a reliability mechanism. It is not evidence that the system has been load-tested or operates as a distributed service.

Dashboard workflow map

The browser interface organizes the platform into eight functional views. It uses the same API contracts as other clients and renders numerical signal summaries locally. The following map describes implemented responsibilities, not a claim of completed browser usability testing.

Typical analysis path

Start with Data Library to establish input provenance, then use Experiments to train and inspect a model. Signal Lab supports recording analysis and controlled fixture exploration. Spoofing analysis applies a registered telemetry model, while Drift examines a batch against its training reference.

Review path

Overview summarizes stored records and recent activity. Incidents supports investigation status and evidence export. Sources identifies external data resources and integration notes so a source can be assessed before registration.

Presentation boundaries

The interface does not show an invented live receiver fleet. Synthetic and replay origins remain relevant to interpretation. Spectrum and spectrogram values are generated by the processing API; they are not measurements from attached hardware.

View	User task
Overview	Inspect registry counts and recent activity.
Signal Lab	Upload or simulate I/Q and inspect spectral evidence.
Spoofing	Apply telemetry classification and inspect output.
Data Library	Register, preview and retrieve supported datasets.
Experiments	Submit training and inspect model reports.
Drift	Compare observation batches with training references.
Incidents	Review evidence and update status.
Sources	Inspect external dataset links and integration context.

> Browser visual and interactive verification was not completed in the development record. API tests do not substitute for that missing verification.

Security and failure handling

The security controls focus on a local research application with explicit input boundaries. They reduce common failure modes without providing the identity, network isolation or audit guarantees of a managed multi-user service.

Access controls

SENTINEL_API_KEY enables a constant-time comparison for API requests. The CLI rejects a non-loopback bind without a configured key. Write requests carrying an Origin that differs from the application origin are blocked. Requests from command-line clients may omit Origin, so that check is not a replacement for authentication.

Browser and export handling

Responses include content-type sniffing protection, frame denial and a same-origin referrer policy. CSV export prefixes dangerous spreadsheet formula-leading text. These controls protect specific presentation paths; they do not sanitize the scientific meaning of an imported observation.

Safe artifact assumptions

Joblib files must remain trusted. Checksums detect replacement relative to trusted metadata and local-path checks constrain artifact lookup. The application does not accept user-uploaded serialized estimators. Compromise of the local registry and filesystem remains outside that integrity check.

Deployment gaps

The service has no role hierarchy, per-user authorization, built-in TLS or cryptographically signed evidence. For broader access, authentication, transport protection, request-rate controls and audit retention need deliberate design and verification. API documentation and health endpoints are not behind the API-key gate.

Failure class	Implemented response
Malformed data	Reject before analysis or training.
Changed artifact	Fail checksum verification.
Foreign browser write origin	Reject the write request.
Training queue saturation	Return a bounded-capacity rejection.

> Avoid treating operational alerts as authenticated evidence of an attacker. The review workflow needs independent acquisition context.

Installation and deployment

The core package can be installed into an isolated Python environment from the repository root. The server defaults to loopback port 8765. Storage is configurable so generated datasets and models can be retained separately from source files.

Local environment

Use a Python version supported by the project dependencies. Install the editable package with development extras for testing. Add the vision extra only when using image training. On Windows, a shorter environment path can avoid path-length problems with large optional dependency trees.

Configuration

SENTINEL_DATA_DIR selects persistent runtime storage. SENTINEL_API_KEY enables API authentication. Keep credentials outside committed source. Use the same data directory consistently when importing, training and starting the server so those commands resolve the same registry.

Container definition

The Dockerfile describes a Python 3.12 slim image running as a non-root user. Compose binds the service locally, uses persistent storage and requires an API key. These files are provided as deployment definitions; the development record does not include a successful container build or execution.

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -e ".[dev]"
gnss-sentinel serve

# Optional image workflow dependencies
python -m pip install -e ".[vision]"

# Separate storage for a controlled experiment
gnss-sentinel --data-dir runtime-study demo --seed 42
gnss-sentinel --data-dir runtime-study serve
```

> The default URL is http://127.0.0.1:8765. A network-accessible deployment needs transport and access controls beyond the local launch command.

Verification strategy and evidence

The development verification record reports 30 passing tests, including the optional ResNet18 smoke test. It also records successful Python compilation, JavaScript syntax checking, Git whitespace checks and API server startup. This report summarizes that evidence; it does not claim a new full-suite execution.

Contract testing

Tests target invalid signals, finite feature outputs, aligned plot axes and deterministic generation. Evaluation tests check group separation and training-only scaler statistics. Integrity tests exercise changed dataset and model content, while inference tests confirm reload behavior and out-of-range abstention.

Service integration

API tests exercise import, asynchronous training, model reporting, prediction, incident creation and resolution, export and drift routes. Access tests cover API keys and foreign-origin writes. The vision smoke test exercises one epoch on random images and checkpoint prediction.

Evidence limits

No public-dataset benchmark, physical receiver trial, GPU training experiment or external load test was performed. Browser visual inspection and Docker execution were not completed. A hosted CI workflow exists but its execution is not part of the recorded evidence.

Verification layer	Recorded outcome	What it establishes
Automated suite	30 passed	Exercised contracts and workflows.
Signal fixtures	Six classes covered	Numerical processing behavior.
Artifact tests	Reload and tamper cases	Persistence and integrity checks.
Vision smoke	One epoch; random images	Executable training and reload path.
Synthetic experiments	Both tasks completed	Model-selection workflow executes.
Browser / Docker	Not verified	No visual or container-run claim.

> A test count measures executed checks, not scientific validity. Performance claims require independent, representative data and a documented evaluation protocol.

Reproduction and operating runbook

Reproduction begins by fixing the source data, feature contract, software environment and split policy. A seed alone is insufficient when source content, dependency behavior or acquisition groups change.

Reproduce the fixture workflow

Use a fresh runtime directory, install the documented dependencies and run demo with seed 42. The command registers fixture datasets and fits both task models. It can reuse previously registered fixture content in the same runtime, so a new directory is important when intentionally changing fixture generation.

Evaluate an imported dataset

Review labels, units and group identifiers before import. Record the returned dataset identifier and fingerprint, then train against that identifier. Inspect partition counts and class support before interpreting the score. Preserve the CSV, splits, model report and environment versions as one experiment package.

Investigate a failed run

Check validation errors first, followed by group coverage and artifact integrity. Do not bypass a grouped split failure by deleting group metadata. After a process interruption, inspect the failed job and resubmit deliberately. Restore storage only from a coordinated database-and-files snapshot.

Retain a trustworthy record

Document all preprocessing outside the application, including filtering, label remapping and I/Q conversion. Keep held-out captures isolated from repeated design decisions. When introducing new features or thresholds, retain the previous report for comparison instead of replacing its metrics.

```
gnss-sentinel --data-dir study import data.csv --name "Study"
gnss-sentinel --data-dir study train DATASET_ID --seed 42

# PowerShell: opt in to the image smoke test
$env:SENTINEL_TEST_VISION = '1'
python -m pytest -q
```

> This example uses the default spoofing task and a separate study directory. Replace DATASET_ID with the identifier returned by import.

Limitations and validity assessment

The implemented platform is broader than an individual classifier, but its current empirical evidence remains limited. Strong evaluation requires separating software correctness, fixture separability and generalization to independently acquired observations.

Internal validity

The pipeline isolates preprocessing and preserves a test partition. Exact duplicates and changed artifacts are checked. However, human decisions based on repeatedly observed test scores can still bias an experiment. Group names also need external justification; syntactically disjoint identifiers alone do not prove independent acquisition.

External validity

The telemetry fixture encodes statistical patterns rather than a validated receiver model. The clean signal fixture is noise-only, and generated groups are not field sessions. Public recordings, different receivers and changing acquisition conditions have not been benchmarked. The reported high scores must remain attached to their synthetic provenance.

Construct validity

A predicted waveform family is not the same as a confirmed malicious emitter. An uncertain label, anomaly score or PSI flag describes a different aspect of the observation. The current system does not measure positioning error, navigation integrity or successful mitigation.

Engineering validity

A single process, local SQLite registry and bounded executor form a useful controlled runtime. There are no measured throughput or availability claims. Browser, container and hosted-CI execution gaps remain explicit. Optional image inference is separate from the dashboard registry.

Evidence available	Claim it does not support
Synthetic test macro F1	Field detection probability.
Passing automated tests	Absence of all defects.
Artifact checksum	Authenticity under registry compromise.
Drift threshold crossing	Proof of an attack.
Container files	Verified container deployment.

Technical development roadmap

Further development should first strengthen evaluation, then extend operational scale. Each proposed stage needs an observable acceptance condition. The items below are planned directions and are not described elsewhere as completed features.

1. Recorded-data benchmark

Integrate documented public recordings and receiver telemetry through explicit adapters. Preserve official held-out partitions when available. Establish capture-aware groups, verify licenses and record units. Acceptance requires an auditable manifest, reproducible splits and per-class results on independent captures.

2. Decision calibration

Evaluate probability calibration, abstention frequency and anomaly false alarms with validation data. Measure the combined incident policy separately from the classifier. Acceptance requires a chosen operating point and held-out estimates of false alarms, missed cases and review volume.

3. Robustness and performance

Compare receivers, acquisition settings, signal conditions and class prevalence. Profile memory, processing latency and training duration under defined workloads. Acceptance requires repeat measurements and documented hardware rather than a single fast demonstration run.

4. Shared-service architecture

Move durable jobs into an external queue, introduce shared artifact storage and a server database, and define worker leases and retry semantics. Add authenticated users, authorization and append-only review history. Acceptance requires restart recovery, concurrent-client tests and access-control verification.

5. Integrated analysis

Consider linking image models with the registry and comparing complementary signal representations. Any fusion or temporal model should be evaluated against a fixed baseline with controlled splits. New interfaces should follow verified backend contracts and receive browser interaction tests.

> Scale should be demonstrated through data volume, independent evaluation and measured service behavior. Adding components alone does not establish capacity or reliability.

References and implementation sources

Implementation descriptions in this report are based on the repository modules, saved model reports and development verification record. External references identify dataset entry points and relevant library contracts; their presence does not imply that the datasets were downloaded or benchmarked.

[1] Original research repository

alicmu2024, GNSS Jamming Detection and Classification using Machine Learning, Deep Learning and Computer Vision. The starting notebooks and accompanying documentation are retained for provenance. https://github.com/alicmu2024/GNSS-Jamming-Detection-and-Classification-using-Machine-Learning-Deep-Learning-and-Computer-Vision

[2] Raw complex-signal dataset

Swinney and Woods, Raw IQ dataset for GNSS GPS jamming signal classification. External dataset catalog entry. https://zenodo.org/records/4629685

[3] Additional dataset entry points

GNSS jammer image data: https://zenodo.org/records/3783969. GPS spoofing telemetry: https://data.mendeley.com/datasets/z7dj3yyzt8/1. Formats, licenses and acquisition metadata must be assessed before integration.

[4] Preprocessing and leakage

scikit-learn, Common pitfalls and recommended practices. https://scikit-learn.org/stable/common_pitfalls.html

[5] Short-time Fourier transform

SciPy, scipy.signal.ShortTimeFFT API reference. https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.ShortTimeFFT.html

[6] Image model constructor

Torchvision, resnet18 API reference. https://docs.pytorch.org/vision/stable/models/generated/torchvision.models.resnet18.html

Repository evidence

Source: src/gnss_sentinel and tests. Verification: docs/VERIFICATION.md. Experiment evidence: locally stored report.json and splits.json for the telemetry and signal-feature models. Generated runtime artifacts are excluded from the source archive.

> Abbreviations: GNSS - Global Navigation Satellite System; I/Q - in-phase/quadrature; FFT - fast Fourier transform; STFT - short-time Fourier transform; PSI - population stability index; API - application programming interface.